

Pemrograman Web

Jobshet 03

Nama : Muhammad Ramdhani Ariyadin

Kelas : SIB 2 E

NIM : 254107060018

Link github : <https://github.com/MuhammadRamdhaniAriyadin/Pemrograman-Web>

Soal

1. Menambahkan breakpoint baru @media (min-width: 1400px) pada file style.css untuk memperbaiki tampilan layar monitor yang sangat lebar . Nilai properti max-width pada elemen <main> menyesuaikan 1000px menjadi 1300px agar ruang layar tidak menyisakan area kosong.

```
@media (min-width: 1400px) {  
  main {  
    max-width: 1300px;  
  }  
}
```

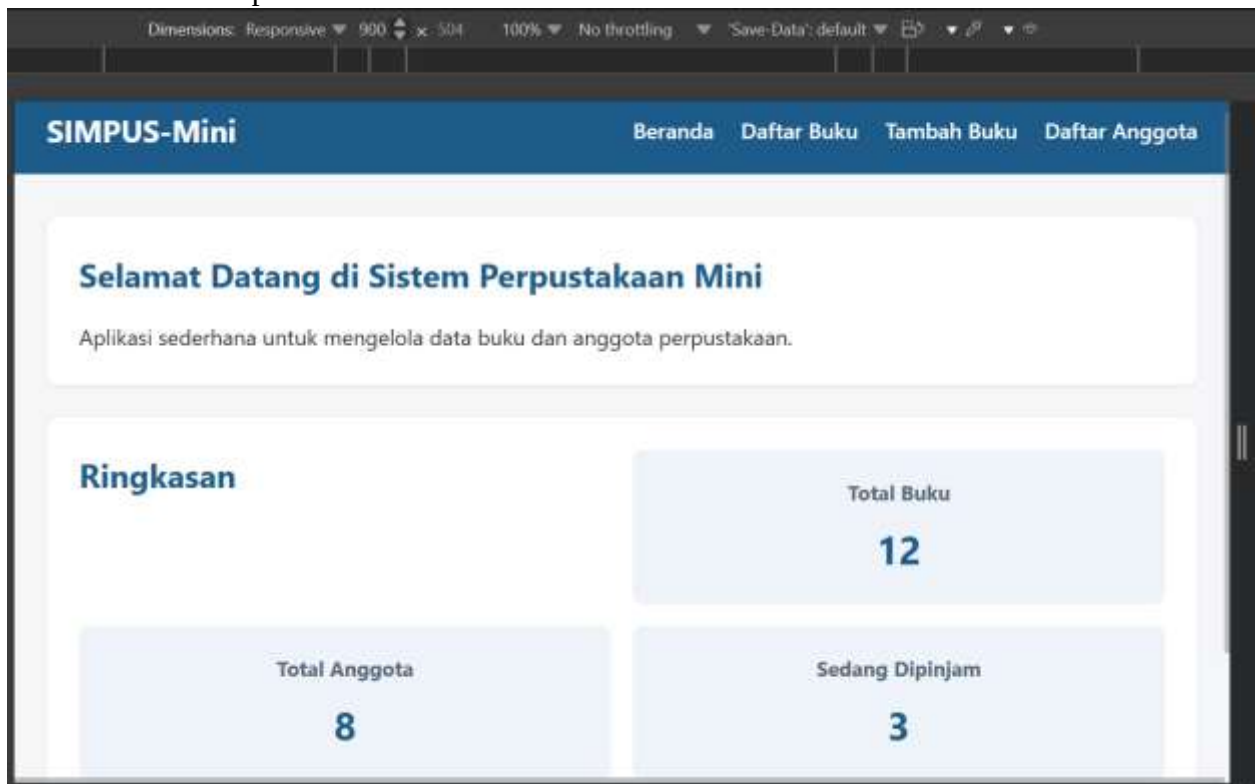
Pembahasan

Penambahan breakpoint @media (min-width: 1400px) bertujuan agar desain web responsif ketika resolusi tinggi. Dan ketika default kontainer utama dibatasi 1000px. Trus pas di buka di monitor lebar (resolusi $\geq$ 1400px), nah query ini otomatis memperluas menjadi 1300px, sehingga tata letak terlihat lebih sesuai.

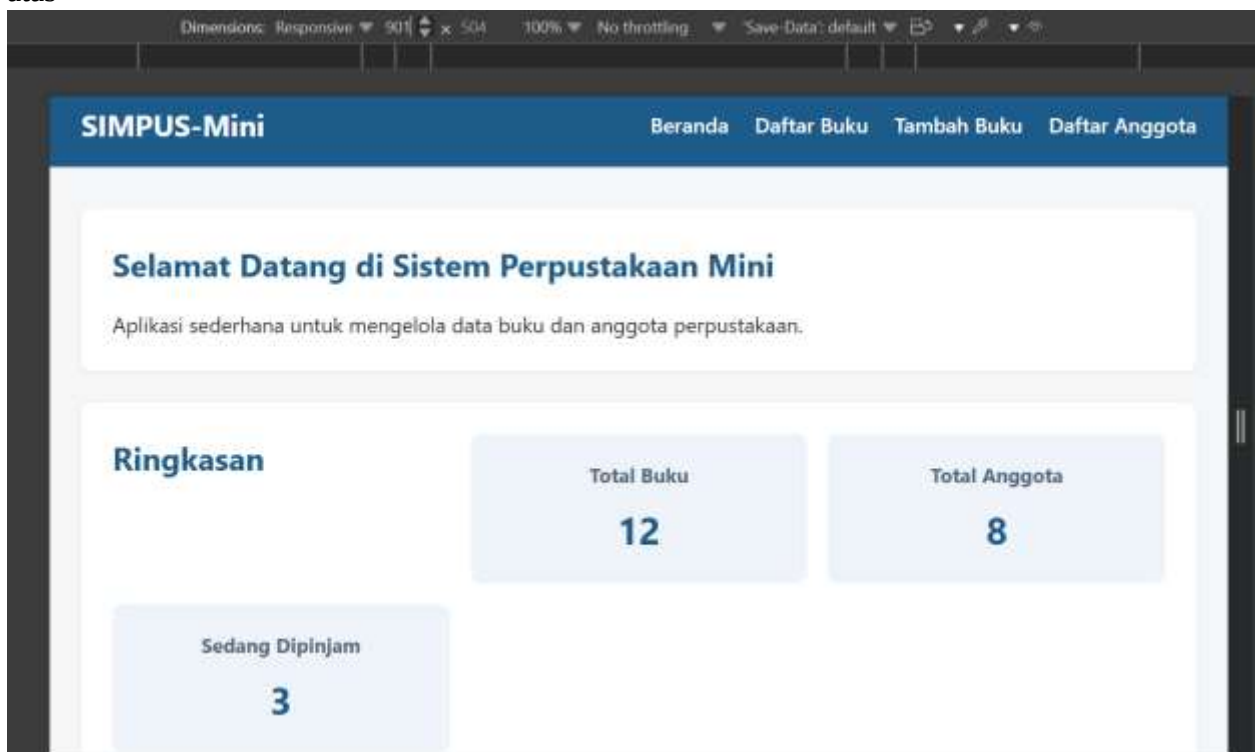
2. Mengubah nilai breakpoint untuk tampilan tablet pada file style.css yang awalnya max-width: 768px menjadi max-width: 900px

```
/* Tablet ke bawah */  
@media (max-width: 900px) {  
  main section:nth-of-type(2) {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}
```

ketika ukuran viewport lebih kecil dari 900 maka satu kolom di atas turun ke bawah



ketika ukuran viewport lebih besar dari 900 maka satu kolom yang bawah kembali ke atas



3. Terapkan pola table-responsive ke elemen lain yang berpotensi melebar di layar sempit, misalnya kalau suatu saat kamu menambahkan blok kode `<pre>` yang panjang di salah satu halaman.

```
.table-responsive,|
pre {
  overflow-x: auto;
  max-width: 100%;
}
```

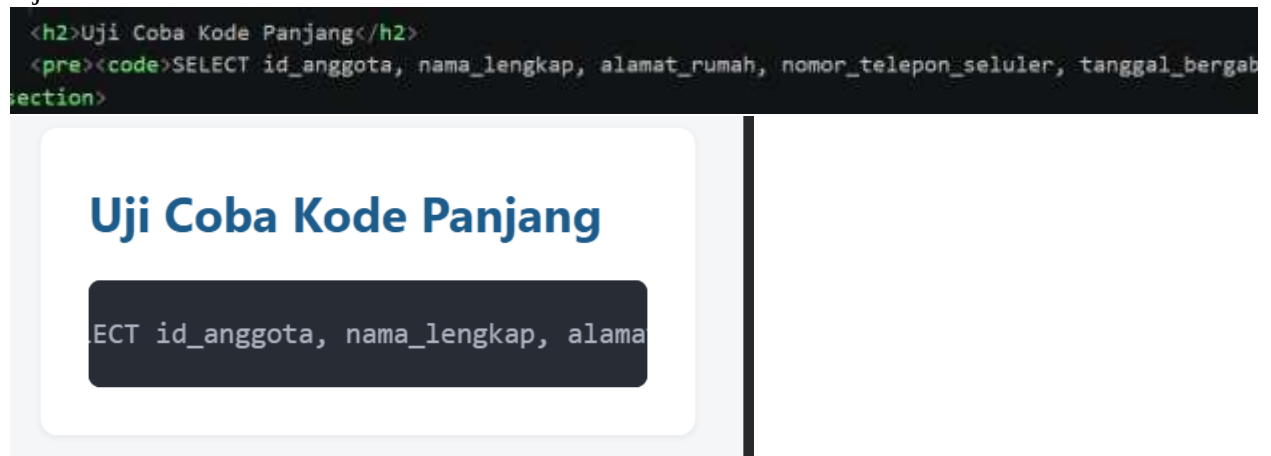
Langkah kerja

Menerapkan pola overflow scroll horizontal yang sebelumnya digunakan pada tabel (.table-responsive) ke elemen `<pre>` pada file assets/css/style.css. Penambahan properti `overflow-x: auto` dan `max-width: 100%` memastikan elemen dengan konten berukuran panjang dapat digulir (scroll) secara horizontal jika melebihi lebar layar.

Pembahasan

Elemen seperti `<pre>` (teks/kode terformat) dan `<table>` memiliki sifat alami tidak dapat mematahkan baris (line-wrap) secara otomatis jika teks di dalamnya terlalu panjang tanpa spasi. Pada layar sempit (mobile), hal ini dapat menyebabkan layout overflow yang merusak tampilan keseluruhan halaman (horizontal blowout). Dengan menerapkan `overflow-x: auto`, elemen tersebut akan terisolasi di dalam kontainernya sendiri dan memunculkan bilah gulir (scrollbar) internal ketika melampaui batas viewport, sehingga seluruh tata letak halaman tetap rapi dan tidak melenceng.

Uji coba



Uji coba dilakukan dengan memasukkan blok kode SQL yang panjang di dalam elemen `<pre><code>` pada halaman web, kemudian dijalankan menggunakan Responsive Design Mode pada ukuran viewport perangkat seluler (375px).

Hasil pengujian menunjukkan bahwa elemen `<pre>` tidak menyebabkan seluruh halaman mengalami horizontal scroll (layout overflow). Sebaliknya, properti `overflow-x: auto` berhasil memunculkan bilah gulir (scrollbar) internal khusus pada area blok kode tersebut, sehingga halaman utama tetap stabil dan kode tetap dapat dibaca secara utuh dengan menggesernya ke samping.

4. Ubah posisi ikon hamburger — misalnya pindahkan `.nav-toggle-label` ke urutan terakhir di `<header>` (setelah `<nav>`) lalu amati apakah sibling combinator `.nav-toggle:checked ~ nav` di bab 3 §3.5 masih bekerja — ingat catatan bahwa combinator `~` mensyaratkan target berada setelah elemen sumbernya di HTML.

Ubah struktur elemen di dalam `<header>` dengan memindahkan `<label for="nav-toggle" class="nav-toggle-label">☰</label>` ke urutan paling akhir (setelah elemen `<nav>`):

```
<h1>SIMPUS-Mini</h1>
<!-- Input checkbox tetap di atas agar combinator -->
<input type="checkbox" id="nav-toggle" class="nav-toggle">
<nav>
  <ul>
    <li><a href="index.html">Beranda</a></li>
    <li><a href="buku/list.html">Daftar Buku</a></li>
    <li><a href="buku/tambah.html">Tambah Buku</a></li>
    <li><a href="anggota/list.html">Daftar Anggota</a></li>
  </ul>
</nav>
<!-- Ikon hamburger dipindahkan setelah <nav> -->
<label for="nav-toggle" class="nav-toggle-label">&#9776;</label>
</header>
```

Langkah kerja

Memindahkan posisi elemen `<label class="nav-toggle-label">` (ikon hamburger) di dalam `<header>` sehingga berada setelah elemen `<nav>`. Pengujian dilakukan pada ukuran viewport perangkat seluler (`480px`) untuk menguji fungsionalitas tombol menu serta keabsahan aturan subsequent sibling combinator (`~`).

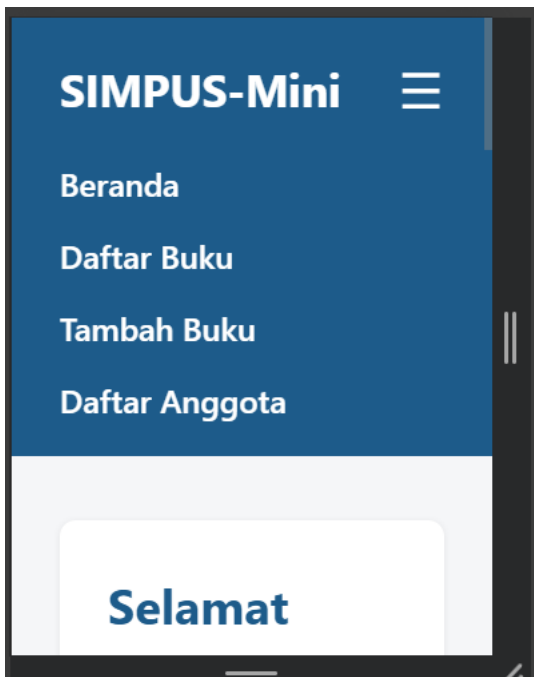
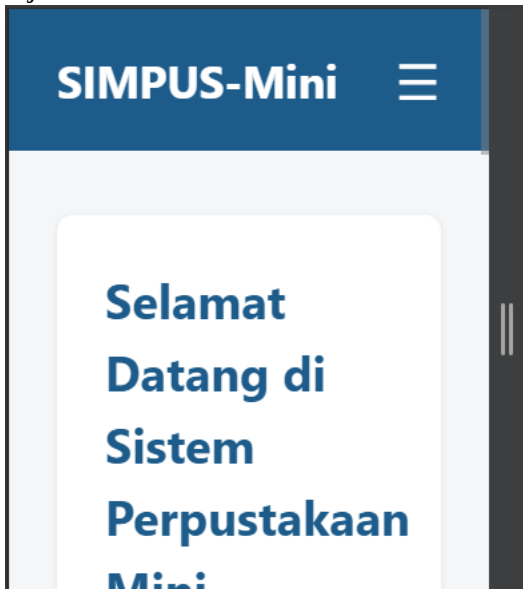
Pembahasan

Hasil eksperimen menunjukkan bahwa menu navigasi tetap bekerja secara normal meskipun posisi `<label>` dipindahkan ke urutan paling akhir di dalam elemen `<header>`. Hal ini terjadi karena penentu utama pemicu (trigger) pada CSS checkbox hack adalah keberadaan elemen `<input type="checkbox" id="nav-toggle">`.

Aturan CSS `.nav-toggle:checked ~ nav` mengevaluasi hubungan antara elemen `<input>` (sumber) dan elemen `<nav>` (target). Karena `<input>` masih berada sebelum `<nav>` dalam urutan struktur DOM HTML, syarat subsequent sibling combinator (`~`) tetap terpenuhi.

Elemen `<label>` hanya berfungsi sebagai pemantik klik melalui atribut `for="nav-toggle"`, sehingga posisinya tidak memengaruhi logika relasi CSS tersebut.

Uji coba



- Perhatikan posisi ikon hamburger (`☰`) yang kini berpindah ke sisi paling kanan dari `<header>`.
- Klik ikon hamburger tersebut untuk membuka dan menutup menu navigasi.
- Hasil Pengamatan: Menu navigasi (`<nav>`) tetap berfungsi dengan normal (muncul dan sembunyi saat ikon diklik).

5. Bandingkan dengan pendekatan mobile-first — coba tulis ulang style.css dari nol memakai @media (min-width: ...) alih-alih max-width, dan rasakan sendiri bedanya alur berpikirnya.

Langkah kerja

Menulis ulang seluruh berkas assets/css/style.css menggunakan pendekatan Mobile-First Design. Aturan CSS utama ditulis tanpa @media khusus untuk melayani tampilan layar kecil (seluler) secara mendasar. Selanjutnya, digunakan teknik progressive enhancement menggunakan media query @media (min-width: ...) secara bertahap untuk menyesuaikan tata letak pada perangkat dengan layar yang lebih lebar (tablet, desktop, hingga ultra-wide).

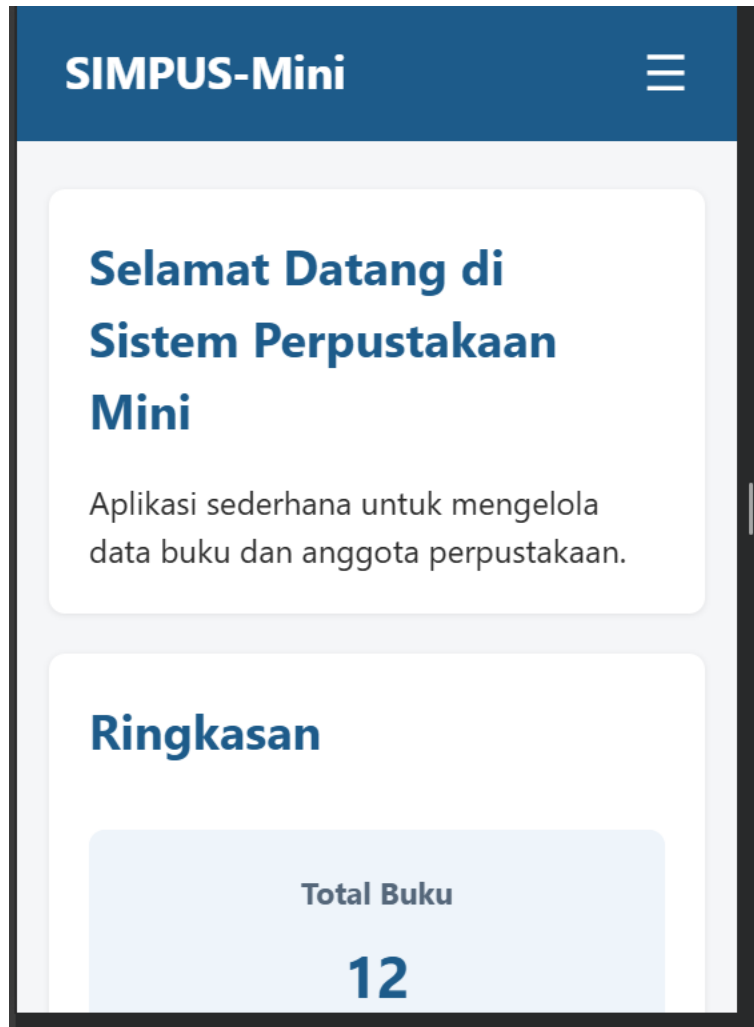
Pembahasan

Perbedaan utama dari alur berpikir Desktop-First (max-width) dan Mobile-First (min-width) terletak pada titik awal penyusunan gaya (styling baseline):

- Alur Berpikir & Efisiensi Kode: Pada Mobile-First, gaya dasar dirancang dari kondisi paling minimalis (1 kolom, menu tersembunyi, kontainer fleksibel 100%). Ketika viewport bertambah lebar, properti ditambahkan atau ditingkatkan secara bertahap (progressive enhancement). Hal ini menghilangkan kebutuhan untuk melakukan "pembatalan" properti kompleks yang sering terjadi pada pendekatan Desktop-First.
- Performa Perangkat Seluler: Berkas CSS diproses lebih efisien pada perangkat seluler karena peramban smartphone langsung mengeksekusi blok kode utama tanpa perlu menyeleksi atau mengabaikan blok-blok media query untuk layar lebar.
- Fleksibilitas Desain: Pendekatan min-width mendorong pola pikir pengembangan web modern yang berfokus pada fleksibilitas konten dasar sebelum menambahkan kompleksitas tata letak seperti multi-column grid atau horizontal navigation.

Uji coba

- a. Ukuran Seluler (360px): Peramban mengeksekusi style dasar secara langsung. Tampilan kartu statistik tersusun 1 kolom vertikal dan navigasi tersembunyi dalam mode hamburger menu.





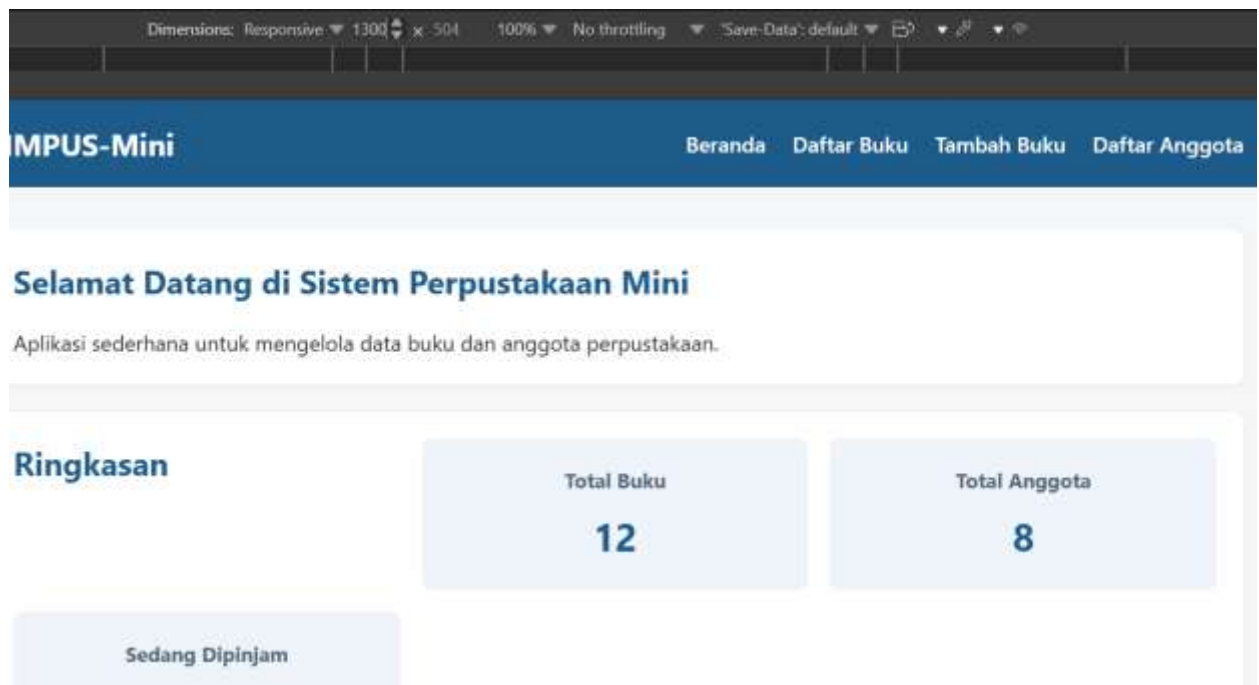
- b. Ukuran Tablet (500px): Ketika viewport mencapai $\geq 481\text{px}$, aturan `@media (min-width: 481px)` aktif dan merubah tata letak kartu statistik menjadi 2 kolom.



- c. Ukuran Komputer/Desktop (900px): Saat viewport mencapai $\geq 769\text{px}$, aturan @media (min-width: 769px) aktif, menghilangkan tombol hamburger, menampilkan navigasi horizontal, serta mengubah kartu statistik menjadi 3 kolom.



- d. Ukuran Monitor Lebar (1500px): Pada viewport $\geq 1400\text{px}$, aturan @media (min-width: 1400px) memperluas lebar kontainer utama (main) dari 1000px menjadi 1300px



Jobsheet 4

1. Perancangan UI

a. Deskripsi Antarmuka

Perancangan wireframe ASCII halaman "Registrasi Anggota Baru" dibuat untuk aktor Tamu agar dapat mendaftar anggota secara mandiri. Tampilan disusun konsisten mengikuti struktur header, navigation, main content, dan footer SIMPUS-Mini.

b. Elemen UI

Formulir memuat input: Nama Lengkap, No. HP, Email, dan Alamat. Field utama diberi atribut required untuk validasi data. Tersedia tombol Daftar Sekarang (submit) dan Batal (kembali ke beranda).

c. Responsivitas

Tata letak disesuaikan dengan style.css menggunakan tampilan block-level input serta toggle navigation menu agar responsif di perangkat mobile maupun desktop.

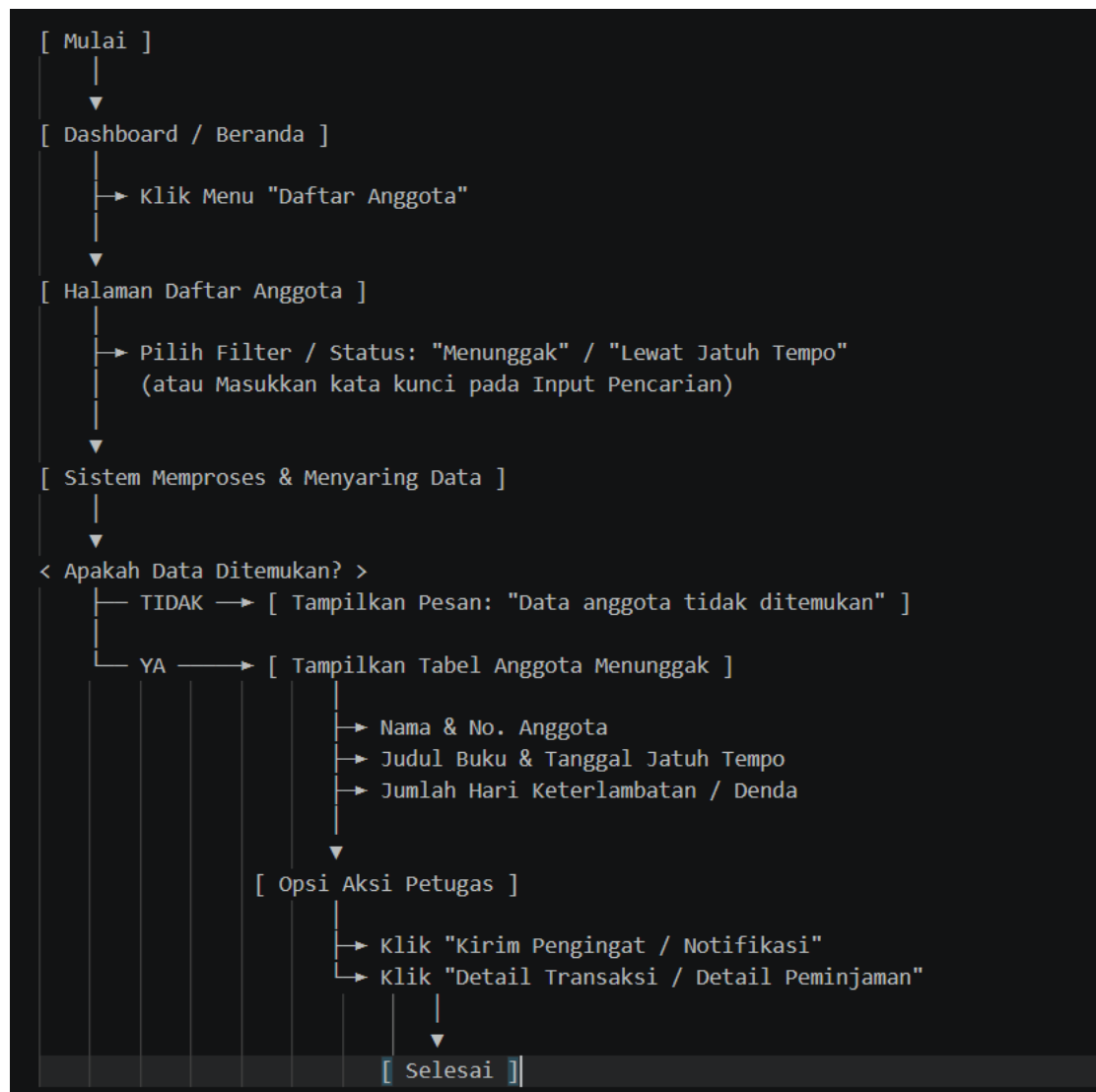
```
+-----+
| SIMPUS-Mini                                     [=] Menu |
+-----+
| [Beranda] [Daftar Buku] [Daftar Anggota] [Registrasi Anggota] |
+-----+
|
| Registrasi Anggota Baru
| Isi formulir di bawah ini untuk mendaftar sebagai anggota resmi.
|
+-----+
| Nama Lengkap *                                  |
| [_____] |
|
| Nomor HP / WhatsApp *                          |
| [_____] |
|
| Email *                                         |
| [_____] |
|
| Alamat Lengkap                                |
| [_____] |
| [_____] |
|
| [ Batal ]                                     [ Daftar Sekarang ] |
+-----+
|
+-----+
| (c) 2026 SIMPUS-Mini -- Jobsheet 3 |
+-----+
```

2. Deskripsi User Flow

User flow ini menggambarkan alur kerja petugas perpustakaan dalam mengidentifikasi dan menangani anggota yang memiliki tunggakan peminjaman buku melewati batas jatuh tempo. Alur dimulai dari halaman utama hingga ditemukannya data anggota yang bersangkutan.

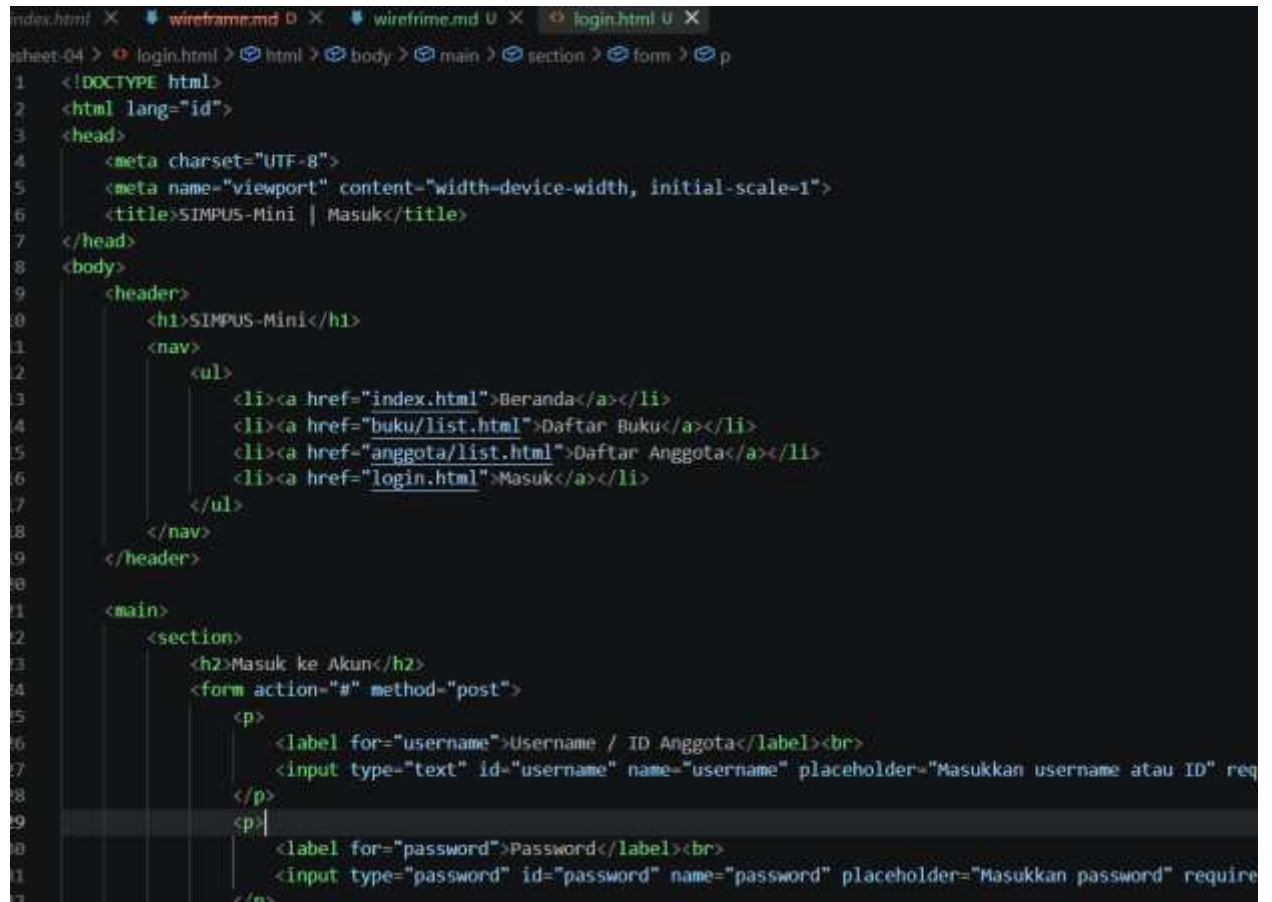
Tahapan Alur Interaksi

Inisiasi: Petugas mengakses menu Daftar Anggota dari navigasi utama SIMPUS-Mini.
Pencarian & Penyaringan: Petugas memanfaatkan fitur filter status (misal: "Lewat Jatuh Tempo") atau memasukkan parameter pencarian khusus Hasil & Aksi: Jika data ditemukan, sistem menampilkan tabel daftar anggota menunggak beserta rincian keterlambatannya. Petugas dapat memilih aksi lanjutan seperti melihat detail transaksi peminjaman atau mengirimkan pesan pengingat (reminder).



3. Daftar edge cases tambahan
 - a. Transaksi dan peminjaman buku
 1. Peminjaman Ganda (Duplicate Loan): Petugas mencoba meminjamkan eksemplar buku yang sama (atau judul yang sama) ke anggota yang sama dua kali berturut-turut sebelum buku pertama dikembalikan.
 - Ekspektasi Sistem: Sistem menolak transaksi dan menampilkan pesan error ("Anggota ini sedang meminjam buku yang sama").
 2. Peminjaman saat ada tunggakan
Petugas mencoba meminjamkan buku baru kepada anggota yang memiliki tunggakan denda atau buku yang belum dikembalikan melewati jatuh tempo.
 - Ekspektasi Sistem: Tombol peminjaman terkunci (disabled) dan sistem meminta penyelesaian tunggakan terlebih dahulu.
 3. Stok buku habis
Buku dicoba untuk dipinjam saat indikator stok bernilai 0.
 - Ekspektasi Sistem: Transaksi diblokir secara otomatis dan tombol diproses menjadi abu-abu (disabled).
 - b. Pengelolaan data dan input form
 1. Penyuntingan/Penghapusan Anggota Aktif:
Petugas mencoba menghapus (Delete) data anggota yang masih memiliki transaksi peminjaman aktif.
 - Ekspektasi Sistem: Sistem mencegah penghapusan data dan menampilkan dialog peringatan ("Tidak dapat menghapus anggota yang masih meminjam buku").
 2. Format Data Tidak Sesuai (Duplikasi / Invalid): Registrasi anggota menggunakan Nomor HP, Email, atau ID Anggota yang sudah terdaftar di database.
 - Ekspektasi Sistem: Form menolak submit dan memberikan highlight merah pada input dengan pesan "Data sudah digunakan"

4. Coba implementasikan wireframe Login sebagai HTML statis (tanpa logika login sungguhan, mirip form Tambah Buku yang belum diproses di jobsheet-01) sebagai latihan menerjemahkan wireframe ke kode nyata — gunakan pola `<label> + <input>` yang sudah kamu kuasai dari dokumentasi jobsheet-01, ditambah satu `<input type="password">` baru untuk field Password.



```
index.html X wireframe.md X wireframe.md U X login.html U X
sheet-04 > login.html > html > body > main > section > form > p
1 <!DOCTYPE html>
2 <html lang="id">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>SIMPUS-Mini | Masuk</title>
7 </head>
8 <body>
9   <header>
10    <h1>SIMPUS-Mini</h1>
11    <nav>
12      <ul>
13        <li><a href="index.html">Beranda</a></li>
14        <li><a href="buku/list.html">Daftar Buku</a></li>
15        <li><a href="anggota/list.html">Daftar Anggota</a></li>
16        <li><a href="login.html">Masuk</a></li>
17      </ul>
18    </nav>
19  </header>
20
21  <main>
22    <section>
23      <h2>Masuk ke Akun</h2>
24      <form action="#" method="post">
25        <p>
26          <label for="username">Username / ID Anggota</label><br>
27          <input type="text" id="username" name="username" placeholder="Masukkan username atau ID" req
28        </p>
29        <p>
30          <label for="password">Password</label><br>
31          <input type="password" id="password" name="password" placeholder="Masukkan password" require
32        </p>
33      </form>
34    </section>
35  </main>
36</body>
37</html>
```

SIMPUS-Mini

- [Beranda](#)
- [Daftar Buku](#)
- [Daftar Anggota](#)
- [Masuk](#)

Masuk ke Akun

Username / ID Anggota

Password

© 2026 SIMPUS-Mini — Jobsheet 3

Penjelasan Elemen Utama

Pola `<label> + <input>`: Mengikuti standar form pada `tambah.html`, di mana setiap input dibungkus dalam paragraf (`<p>`) dengan break line (`<br>`) setelah label.

`<input type="password">`: Elemen baru yang menyembunyikan karakter karakter password demi keamanan tampilan visual.

Atribut `required` & `placeholder`: Memberikan petunjuk visual dan validasi bawaan browser sebelum form dikirimkan.